



Web: inceptez.com Mail: info@inceptez.com Call: 7871299810, 7871299817

Databricks: Lakeflow Declarative Pipelines (Delta Live Tables)

DLT - Append Flow API

The **Append Flow API** in Delta Live Tables (DLT) is a powerful feature used to write to a single target table from multiple source streams or to perform historical backfills without needing a full refresh of the entire pipeline.

Handling Batch vs. Streaming

Append flows are unique because they can handle both types of data within the same target table:

- **Streaming Source:** By default, if the function returns a `readStream`, DLT treats it as a continuous flow.
- **once=True (Batch Mode):** If you are doing a historical backfill from a static folder, you can use `@dlt.append_flow(..., once=True)`. This allows the flow to run once, append its data, and then stop, while other streaming flows continue.

```
import dlt
```

```
dlt.create_streaming_table("combined_table")
```

```
@dlt.append_flow(target="combined_table")
```

```
def table1():
```

```
    return spark.readStream.table("source1")
```

```
@dlt.append_flow(target="combined_table")
```

```
def table2():
```

```
    return spark.readStream.table("source2")
```

DLT – Auto CDC Flow API

Delta Live Tables (DLT) provides the `apply_changes` API to simplify **Change Data Capture (CDC)**. It allows you to ingest a stream of changes (Inserts, Updates, Deletes) and automatically merge them into a target table, ensuring data integrity.

1. Create the Target Table

```
dlt.create_streaming_table("customers_silver")
```

2. Define the CDC Flow

```
dlt.apply_changes(
    target = "customers_silver",
    source = "customers_bronze_stream",
    keys = ["customer_id"],
    sequence_by = F.col("update_time"),
    apply_as_deletes = F.col("op_code") == "D",
    except_column_list = ["op_code"], # We don't need 'I' or 'U' in the final table
    stored_as_scd_type = 1
)
```

DLT – Auto CDC Flow API

Parameter	Type	Description
target	String	The name of the table being updated.
source	String	The name of the streaming source (Bronze/Input table).
keys	List	The primary keys (e.g., product_id) used to identify unique rows.
sequence_by	Column	A timestamp or version number used to handle out-of-order data (the highest value "wins").
apply_as_deletes	Column/Expr	A condition (e.g., col("op") == "D") specifying when a row should be deleted.
apply_as_truncates	Column/Expr	A condition to truncate the target table.
except_column_list	List	Columns in the source that should NOT be included in the target (e.g., metadata/op codes).
stored_as_scd_type	Int	1 for overwriting (current state) or 2 for historical tracking.
track_history_column_list	List	For SCD Type 2, only trigger a new history row if these specific columns change.

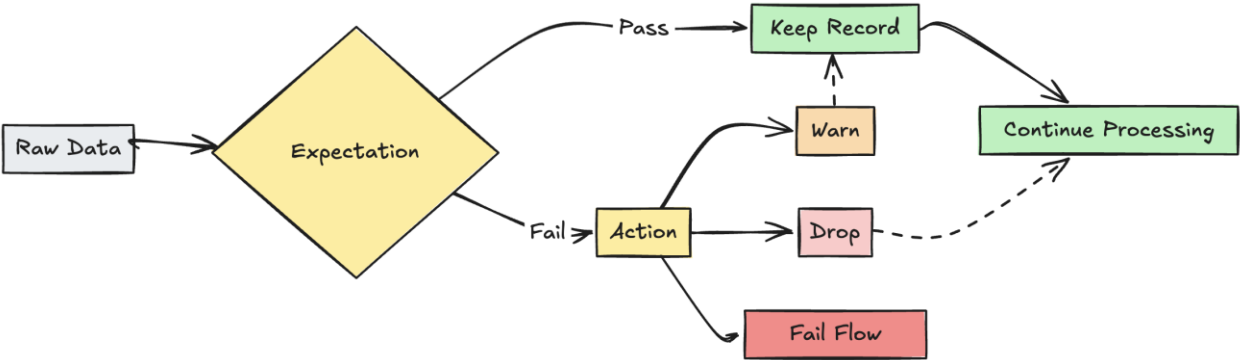
DLT – Data Validations

In **Databricks DLT**, **data validation** is implemented using **Expectations**.

Expectations are optional data quality rules defined in materialized views, streaming tables, or views, using SQL Boolean conditions to validate each record.

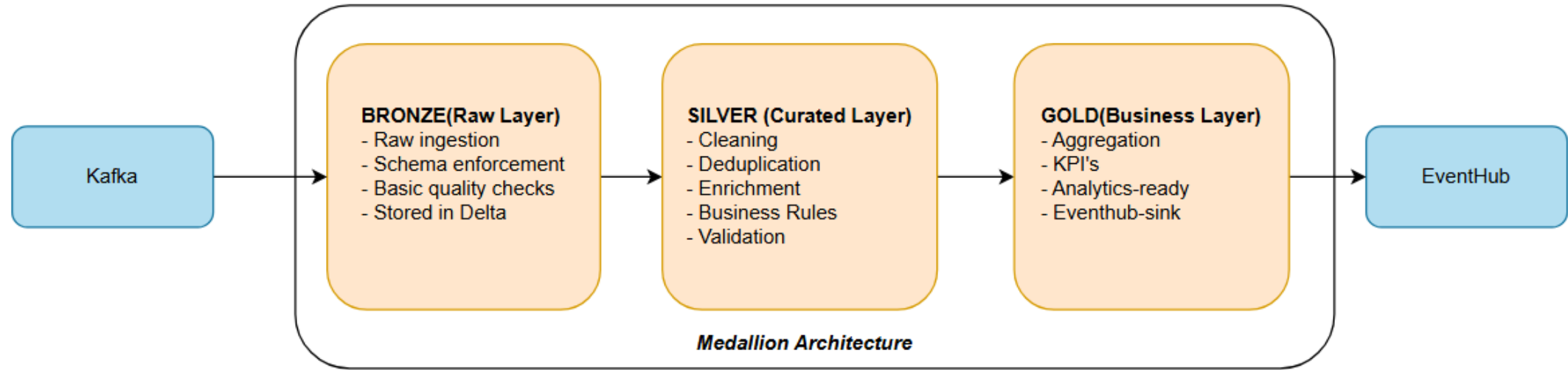
Types of Data Validation in DLT

Type	Behavior	Use Case
EXPECT	Logs bad records	Monitoring
EXPECT_OR_DROP	Drops bad records	Data cleansing
EXPECT_OR_FAIL	Fails pipeline	Critical data(strict rule)



```
@dlt.table(name="sales_multi_validation")
@dlt.expect("valid_amount", "total_amount > 0")
@dlt.expect_or_drop("valid_quantity", "quantity > 0")
@dlt.expect_or_fail("valid_region", "region IS NOT NULL")
@dlt.expect_or_fail(
    "mandatory_fields",
    "sale_id IS NOT NULL AND customer_id IS NOT NULL"
)
def sales_multi_validation():
    return spark.read.table("sales")
```

Fleet Data Management



Business KPIs

- Avg Speed
- Fuel Consumption
- Overheat Count
- Distance Estimation

Payload

```
{
  "vehicle_id": "V1001",
  "timestamp": "2026-01-27T10:12:00Z",
  "speed": 72,
  "fuel_level": 55,
  "engine_temp": 88,
  "latitude": 12.91,
  "longitude": 80.22,
  "driver_id": "D101"
}
```

DLT – Pipeline Modes

In Databricks Delta Live Tables, the **Pipeline Mode** (Continuous vs. Triggered) defines how often data is processed and how compute resources are managed.

Feature	Triggered Mode	Continuous Mode
Logic	Processes all available data once and stops.	Processes data as it arrives, 24/7.
Data Freshness	Minutes to Days (Batch style).	Seconds to Minutes (Streaming style).
Cluster Behavior	Spins up, runs, and shuts down.	Always running.
Cost	Lower (pay only for execution time).	Higher (paying for a 24/7 cluster).
Ideal Use Case	Hourly/Daily reports, Batch ETL.	Real-time dashboards, IoT, Fraud detection.

DLT – Pipeline User Modes

In DLT, **Development Mode** is a configuration designed to speed up the iterative process of writing and testing code. It prioritizes developer speed and cost-saving during the build phase over the strict reliability and retry logic required in production.

Feature	Development Mode	Production Mode
Cluster Reuse	Reuses the same cluster for multiple updates to avoid the 5-10 minute "spin-up" time.	Usually starts a fresh cluster (or uses a clean state) to ensure no "leaked" memory or side effects.
Cluster Lifetime	The cluster stays alive for a set time (default 2 hours) after a run so you can start the next test instantly.	The cluster shuts down immediately after a task finishes (in triggered mode) to save costs.
Retries	Disabled. If a table fails, it stops immediately so you can see the error and fix it.	Enabled. It will automatically retry on common failures (like transient network issues) to ensure the job finishes.

Automatic Toggle: If you run a pipeline manually from the **Pipeline Editor**, Databricks often defaults it to Development mode. However, if the pipeline is triggered by a **Schedule** or a **Job**, it should always be set to Production mode.